

Received 26 October 2025, accepted 16 November 2025, date of publication 25 November 2025,
date of current version 3 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3636995

RESEARCH ARTICLE

iDEAS: Intelligent DVFS for Energy-Efficient Task Scheduling in Mobile Devices With big.LITTLE Computing Architecture

NIMA SAMADI¹, FARBOD YADOLLAHI¹, AND HAMED SHAH-MANSOURI¹, (Member, IEEE)

Department of Electrical Engineering, Sharif University of Technology, Tehran 1458889694, Iran

Corresponding author: Hamed Shah-Mansouri (hamedsh@sharif.edu)

ABSTRACT Heterogeneous multicore computing architectures have fundamentally transformed the computational potential of modern mobile devices by enabling parallel and energy-aware task execution. Dynamic voltage and frequency scaling (DVFS) techniques further allow mobile devices to adjust their operating frequency based on their workload demands to save energy. Additionally, edge computing offers the potential to offload intensive tasks, further enhancing energy efficiency. However, the highly dynamic and unpredictable conditions of mobile environments complicate the effective use of these advancements. To address this challenge and alleviate the burden of short battery life, we focus on enhancing energy efficiency in mobile devices. In this paper, we propose an intelligent DVFS-based algorithm that distinguishes itself from existing literature by jointly optimizing task scheduling, frequency scaling, offloading, and power allocation through learned policies. To do so, we first formulate an optimization problem that aims to minimize the energy while meeting task deadlines. This problem is a highly complex combinatorial optimization, and we prove its NP-hardness. Moreover, the highly dynamic nature of mobile environments further complicates achieving an optimal solution. To address these challenges and adapt to unpredictable changes, we then develop a deep reinforcement learning (DRL)-based algorithm, called iDEAS, which gradually learns the best policy of decision making. We introduce a cost function for different action choices, using which we derive a reward function for the DRL agent. We further evaluate the performance of iDEAS on two distinct task sets and compare it with two state-of-the-art existing works, one prioritizing energy efficiency and the other compromising energy consumption for fewer task drops. The results show that iDEAS reduces energy consumption by 69% and 75% for less computationally intensive tasks, and by 54% and 72% for more demanding tasks, compared to these methods. Notably, iDEAS achieves these energy savings while reducing task drops by 18% for intensive workloads and almost eliminating task drops for lighter workloads.

INDEX TERMS DVFS, deep reinforcement learning, heterogeneous multicore computing architecture, task scheduling, computation offloading.

I. INTRODUCTION

Modern mobile devices and their rapidly evolving applications have led to a significant increase in computational demands. Emerging applications, such as mobile gaming [3], face recognition [4], augmented and virtual reality [5], extended reality (XR) [6], autonomous systems [7], and

telemedicine [8], rely heavily on compute-intensive operations. As a result, these applications consume substantial amounts of energy and accelerate battery depletion, underscoring the critical need to enhance the computational and energy efficiency of mobile devices. It is essential to properly address this challenge not only for extending battery life and improving user experience but also for supporting the growing complexity and performance demands of next-generation mobile applications.

The associate editor coordinating the review of this manuscript and approving it for publication was Huan Zhou¹.

The introduction of heterogeneous multicore computing architecture by ARM [9], which is called big.LITTLE architecture, has greatly enhanced the overall performance of mobile devices. This architecture uses two types of processors. The big processor provides high computing power while the latter is designed to achieve power efficiency. These can greatly enhance the performance of mobile devices and reduce power consumption if the computing tasks are intelligently assigned to an appropriate core based on the requirements of tasks and the availability of processors.

Another promising technology that can enhance the performance of centralized processing units (CPUs) is dynamic voltage and frequency scaling (DVFS), which provides power saving by adjusting the voltage and frequency of the processors dynamically [10]. Although higher CPU frequencies can accelerate task execution, completing tasks well before their deadlines can result in idle periods, known as *slack time*, during which the CPU continues to consume energy. The slack time can be properly adjusted by stepping down or up the CPU's clock frequency using DVFS techniques in order to optimize both energy efficiency and task completion time.

The cloud computing paradigm offers a powerful solution for handling computation-intensive applications by offloading tasks from resource-limited devices. Building on this concept, recent advancements in mobile edge computing (MEC) have brought computation even closer to mobile devices, allowing them to offload demanding workloads to nearby edge servers with reduced latency and improved efficiency. However, offloading every task to edge servers would not necessarily reduce the mobile devices' energy consumption. Moreover, due to the latency in dispatching the tasks, we may even encounter larger processing delays.

A. MOTIVATION

Despite the aforementioned technological advancements, excessive battery consumption remains a critical challenge for mobile devices. To fully leverage these innovations, an efficient task scheduling mechanism is essential to dynamically balance local processing with DVFS optimization and edge offloading. Effective scheduling must determine not only which tasks to offload but also how to distribute the remaining tasks across heterogeneous cores and adjust their frequency settings. Poor scheduling can undermine potential energy savings from DVFS and offloading, hindering the deployment of computation-intensive applications [11]. While existing works employ DVFS to reduce energy consumption, they either overlook the latest developments in heterogeneous multicore processors and the benefits of edge computing or inadequately address the highly dynamic nature of mobile device environments.

Motivated by these limitations, in this paper, we address the following technical questions for mobile devices with big.LITTLE heterogeneous computing architecture.

- 1) *Which computation tasks should be dispatched to the edge server in order to reduce the energy consumption?*

- 2) *How should the tasks be scheduled and allocated between big and LITTLE cores, and what frequency should be set for each core during task execution?*
- 3) *How can these decisions be made intelligently in advance, prior to observing dynamic changes?*

B. CONTRIBUTIONS

To answer the above questions, in this paper, we extend our previous work in [1] and propose an intelligent DVFS algorithm for energy efficient task scheduling and offloading, called iDEAS. We specifically focus on energy efficiency as the primary optimization target because battery limitations remain the most significant constraint for mobile devices running computation-intensive applications. While performance optimization is valuable, energy constraints ultimately dictate the practical utility and user experience of mobile applications. By optimizing energy consumption while maintaining deadline constraints, our approach addresses the most critical bottleneck in MEC environments. Prior studies typically assume a linear inverse relationship between tasks' execution time and CPU frequency. However, modern processors typically exhibit sub-linear scaling due to the memory wall and the influence of static power. The memory wall refers to the widening gap between rapidly increasing processor speeds and comparatively slower memory access times. In this work, we relax this assumption and adopt a general execution-time model that captures arbitrary nonlinear relationships between CPU frequency and execution time. This addresses the limitations of existing studies and provides a more accurate representation of the dynamic behavior of mobile devices. Reinforcement learning (RL) and Q-learning have been used for decision making in dynamic situations (e.g., [1], [12]). However, these are not efficient to handle environments with high-dimensional state spaces. We use deep Q-networks (DQN) that combine traditional Q-learning with deep neural networks (DNNs) to intelligently respond to the unknown and dynamic nature of mobile devices and MEC environments when addressing the limitations of RL-based algorithms. The main novelty of our work lies in enabling mobile devices to exploit the aforementioned advancements (i.e., heterogeneous big.LITTLE architecture, DVFS, and edge offloading) through an algorithm that can actually handle the inherent complexity of the joint optimization problem.

The main contributions of this paper are as follows.

- *Problem Formulation:* We first formulate an optimization problem to jointly make the offloading decision, schedule the tasks and allocate them to the most appropriate processor core, while adjusting its operating frequency. The problem also determines the optimal transmission power for the device's wireless interface when the decision is made to offload the task. The problem objective is to minimize the energy consumption required to execute the tasks, while ensuring each task is executed within its deadline. We formally establish

the NP-hardness of this combinatorial optimization problem, highlighting why exact or conventional optimization methods are impractical in real-time mobile environments.

- *DQN-based Algorithm*: We then propose an algorithm using model-free DQN to learn the best policy of decision making in different situations and make scheduling and offloading decisions. DQN is used for its strong performance, lower computational requirements, and superior generalization to unseen states. In particular, we introduce a cost function that encapsulates both energy consumption and task processing delay, using which we derive a reward function for different action choices. We then use DQN to estimate this reward and identify the most energy-efficient strategies.
- *Complexity Analysis*: We conduct a complexity and memory footprint analysis, demonstrating that the proposed algorithm is lightweight and feasible for mobile deployment. This analysis further underscores the suitability of our DQN-based approach for addressing the inherent complexity of the joint optimization problem.
- *Performance Evaluation*: We investigate the performance of our proposed algorithm and show its superiority in comparison to two existing works in the literature, namely RRLO [13] and DRLDO [14] as well as several baseline schemes. RRLO prioritizes energy efficiency, whereas DRLDO sacrifices energy consumption to achieve fewer task drops. The results demonstrate that our proposed algorithm significantly reduces the energy consumption compared to all these methods. In particular, for computationally intensive tasks, iDEAS reduces energy consumption by 54% and 72% compared to RRLO and DRLDO, respectively, while simultaneously reducing the number of tasks drops. Under light workloads, iDEAS achieves remarkable 69% and 75% reductions in energy consumption compared to RRLO and DRLDO, respectively, while both iDEAS and DRLDO exhibit negligible task drops, and RRLO results in approximately 10% task drop.

This paper is organized as follows. In Section II, we review the related literature. In Section III, we present the system model and formulate the problem. In Section IV, we propose our DQN-based algorithm. In Section V, we evaluate the performance of the algorithm. Finally, Section VI concludes the paper. The implementation of iDEAS is available in [2].

II. RELATED WORK

Recently, numerous studies have investigated energy-efficient task computation on mobile devices by leveraging techniques such as DVFS. In this section, we categorize these works into two groups based on the type of mobile processor and discuss their respective limitations.

A. SINGLE-CORE PROCESSOR

Several related studies focus on mobile devices equipped with single-core processors. In [13], Huang et al. considered

real-time mobile applications and proposed an RL-based algorithm for offloading decisions and DVFS adjustments, aiming to reduce energy consumption. They employed DQN and suggested updating Q-tables randomly, which may lead to sub-optimal decisions. Although this approach targets effective offloading, it does not address task scheduling or processor core allocation. Tang et al. [15] explored task offloading and DVFS in post-disaster MEC networks. They investigated a multi-agent setting in which a mobile device decides whether to offload tasks to a peer or an edge server while adjusting CPU frequency through DVFS. Similarly, Shi et al. [16] examined partial task offloading to fog servers and formulated task offloading and DVFS as an optimization problem. Their objective was to improve energy efficiency, and they proposed an algorithm using decomposition techniques to solve the formulated problem. However, their partial offloading model may not accurately reflect the characteristics of real-world computation tasks.

B. MULTICORE PROCESSOR

Recent research has explored modern computing architectures based on ARM's big.LITTLE model, aiming to reduce the energy consumption of mobile devices. Lv et al. [17] modeled tasks as a graph and proposed an algorithm to reduce delay and enhance task parallelism by leveraging multiple processor cores. However, their approach may not generalize well to more complex tasks. Wang et al. [18] proposed a heuristic task scheduler that categorizes tasks into two types: deadline-prior tasks, which prioritize meeting deadlines even at higher energy costs, and energy-prior tasks, where minimizing energy consumption is the main objective. They suggested transforming some deadline-prior tasks into energy-prior ones to reduce energy usage while still meeting deadlines. In [19], Kim et al. introduced a heuristic DVFS algorithm for task execution on big.LITTLE architectures. They used support vector machines (SVM) to predict task execution time for improved scheduling and frequency setting. However, the SVM model does not account for task dynamics and the inherent randomness of real-world scenarios. Focusing on the big.LITTLE architecture, Li et al. [20] proposed a workload governor tailored for mobile gaming, abstracting game scenes and predicting the computational requirements for each frame. Based on these predictions, CPU frequencies were adjusted using the Linux kernel's DVFS algorithm. In [21], Wang et al. proposed a two-layer hybrid scheduling strategy which involves an initial task allocation step guided by an RL-based algorithm, followed by a dynamic migration algorithm to balance load among homogeneous cores during runtime. The objective is to minimize energy consumption while meeting task deadlines and improving resource utilization on big.LITTLE architecture.

Although the aforementioned studies explored the big.LITTLE computing framework, none considered the possibility of offloading tasks to an edge server. Ki et al.

[22] proposed a DVFS method based on a genetic algorithm, combined with task memory placement and selective offloading. However, by offloading only heavy tasks to the edge server, optimal energy savings are not achieved, as some tasks could be executed locally before their deadlines with lower energy consumption. Geng et al. [23] introduced a heuristic offline algorithm based on DVFS to address task offloading and allocation challenges in big.LITTLE architectures. Their approach recursively assigns tasks to the most appropriate CPU core to minimize energy consumption, but it does not consider dynamic network conditions, which are crucial in real-world scenarios. Rexha et al. [24] focused on object detection tasks on mobile devices with big.LITTLE architecture and proposed a heuristic algorithm for task scheduling and offloading aimed at reducing energy usage. However, their method fails to effectively adapt to environmental dynamics. Panda et al. [14] investigated energy-efficient computation offloading in MEC environments, focusing on mobile devices equipped with multicore processors. However, their work does not account for the big.LITTLE architecture, which is prevalent in modern mobile devices and plays a significant role in energy-performance trade-offs. Moreover, their proposed model assumes that each task can be partitioned into arbitrarily small components following the partial offloading task model. This may not align with the constraints of practical applications, thereby limiting the real-world applicability of their approach.

A preliminary version of this work was presented in [1]. The novelties and contributions of this paper, in comparison to [1], are threefold: (i) We formulate a more comprehensive optimization problem that jointly schedules tasks, adjusts processor operating frequencies, makes offloading decisions, and allocates transmission power. Making all these decisions simultaneously allows us to fully leverage the capabilities of modern mobile devices. Moreover, expanding the optimization space helps minimize energy consumption while still meeting task deadlines. (ii) We incorporate a realistic wireless channel model and optimally allocate transmission power, whereas [1] assumed a fixed transmission rate. (iii) We propose iDEAS using DQN, leveraging the strengths of deep learning. Compared to the Q-table approach used in [1], DQN offers better generalization and achieves lower energy consumption. Additionally, we analyze the computational complexity and memory footprint of iDEAS.

III. SYSTEM MODEL AND PROBLEM FORMULATION

We consider a mobile device with a state-of-the-art multicore CPU featuring big.LITTLE computing architecture that utilizes DVFS. Various computation tasks arrive at the device, each of which can be either handled locally or sent to a nearby edge server. Depending on the task's characteristics and the availability of the processing resources and data transmission rate to the server, each computing resource could outperform others in certain situations. Fig. 1 illustrates such a system. The task scheduler decides whether to assign the tasks to

the local CPU or offload them to the edge server via the wireless interface. When the CPU load is low, the scheduler mainly uses the high-efficiency (i.e., LITTLE) cores within the LITTLE cluster, while when the CPU is under heavy load, the scheduler assigns tasks to high-performance (i.e., big) cores to expedite the task execution.

A. TASK MODEL

Considering that most of the tasks in mobile devices repeat periodically, we utilize the periodic model for tasks as inspired by [13], [25], and [26]. Let N denote the total number of such tasks. Each task $i = 1, \dots, N$ is characterized by tuple $t_i(p_i, b_i, w_i)$ where p_i denotes its period interval and shows how often the task arrives. Moreover, b_i and w_i denote the amount of task input data (in bits) for processing and its worst case execution time (WCET), respectively. WCET is the estimated upper bound for processing the task [13], [26]. We also denote the set of these N tasks by $\mathcal{T}$ as below:

$$\mathcal{T} = \{t_1(p_1, b_1, w_1), t_2(p_2, b_2, w_2), \dots, t_N(p_N, b_N, w_N)\}.$$

Inspired by prior works [13], [14], [26], we consider that all tasks are independent. This allows us to formulate the problem in a tractable manner and focus on the core objective of jointly optimizing DVFS, task scheduling, and computation offloading.

We further introduce the utilization of each task i , which is the ratio between its WCET and its period and is defined as

$$u_i \triangleq \frac{w_i}{p_i}, \quad (1)$$

and the total utilization of task set $\mathcal{T}$ which is

$$U_{\mathcal{T}} \triangleq \sum_{\forall i \in \mathcal{T}} u_i. \quad (2)$$

B. CPU MODEL AND DVFS

The big.LITTLE architecture is one of the most popular multicore CPU architectures that is widely used in modern mobile devices. This is a heterogeneous architecture developed by ARM that has two kinds of cores: the big cores which are fast but power-hungry and use higher frequencies to be able to provide higher performance for computation-intensive tasks. In contrast, the LITTLE cores are slow and optimized for performing light tasks with lower energy consumption. We should allocate tasks to these two types of cores dynamically and adjust their core frequencies based on the task's characteristics.¹ DVFS is widely used for CPU frequency adjustment. It reduces power consumption by adjusting CPU voltage and frequency. DVFS techniques are based on reducing the time that the CPU is idle by reducing

¹ Similar to [13], [14], [17], [18], [19], [20], [21], [22], and [23], we assume that once a task is allocated to a CPU core, it remains on that core and is not migrated during execution. Although the big.LITTLE architecture supports task migration to enhance performance, this assumption allows us to focus on energy-aware task scheduling, DVFS, and computation offloading, while still capturing the key operational dynamics of heterogeneous computing platforms.

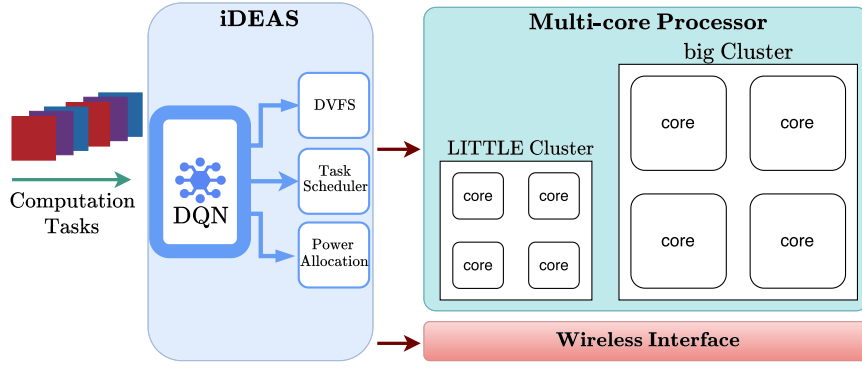


FIGURE 1. Overview of the system architecture. The big cores together form the big cluster and the LITTLE cores belong to the LITTLE cluster. When the system load is light, only the LITTLE cluster is used while in the high load, the big cluster is also involved in executing the tasks.

the frequency. It may also increase the frequency to prevent tasks from missing their deadlines and ensure efficient processor utilization. DVFS techniques are introduced in [10] and their energy model and characteristics are discussed in [27] and [28].

At each moment, the operating frequencies of CPU cores are chosen from possible frequency values by DVFS. We denote the set of these frequencies by $\mathbf{f} \in \{f_1, \dots, f_m\}$ where m is the number of possible different frequencies, which depend on the CPU and varies among different vendors. Without loss of generality, we assume that $f_1 < f_2 < \dots < f_m$. Each frequency $f \in \mathbf{f}$ corresponds to an operational voltage which we denote as $V^b(f)$ and $V^L(f)$ for big and LITTLE cores, respectively. Note that these cores may operate on different frequencies at each moment.

C. COMMUNICATION MODEL

When a task is designated for offloading, it is transferred via the wireless interface to the edge server, which then executes the task and sends the results back. Similar to [29], we assume the communication channel follows the quasi-static block fading model, meaning that the channel remains constant during the offloading, while it may change from time to time. Let q_i and h_i denote the transmission power of the mobile device and the channel gain, respectively, when offloading task t_i . We also let σ denote the noise power. The channel power gain and noise are modeled as Rayleigh and Gaussian distributions, respectively. According to the Shannon formula, the uplink rate, denoted by r_i , can be expressed as

$$r_i = W \log_2 \left(1 + \frac{q_i h_i}{\sigma} \right), \quad (3)$$

where W denotes the channel bandwidth. We also define the vector $\mathbf{q} = (q_1, \dots, q_N)$ as the transmission power profile of all tasks. Note that, consistent with prior studies such as [13], [14], [30], [31], [32], and [33], we assume that the result returned from the edge server incurs negligible communication overhead. This assumption is valid for the workloads considered in this paper (e.g., classification labels,

decision variables), where the output size is significantly smaller than the input data.

D. ENERGY MODEL

Our objective is to minimize the total energy consumption of the mobile device. The energy consists of two terms: the energy that the device uses to execute the tasks locally, and the communication energy to dispatch the tasks to the edge server if the device arrives at the offloading decision.

1) LOCAL COMPUTATION

The energy consumed in the case of local computing consists of the following two parts: Static power which is comprised of leakage energy consumption, and the dynamic power which is consumed when executing task instructions. Given f as the operating frequency, the static power of big and LITTLE cores can be computed as follows

$$P_s^b(f) = V^b(f)I_s^b, \quad (4)$$

$$P_s^L(f) = V^L(f)I_s^L, \quad (5)$$

where I_s^b and I_s^L denote the big and LITTLE processors' sub-threshold currents, respectively [26], [34]. The processor's sub-threshold current is calculated based on fabrication techniques. Moreover, we define the dynamic power consumption of big and LITTLE cores as $P_d^b(f)$ and $P_d^L(f)$, which can be calculated as follows.

$$P_d^b(f) = Cf^b \left(V^b(f) \right)^2, \quad (6)$$

$$P_d^L(f) = Cf^L \left(V^L(f) \right)^2, \quad (7)$$

where C is the switching capacitance of the transistor gates, which directly depends on the fabrication technology [26]. Now, to calculate the total energy consumption, we need to determine the execution time of each task. Let τ_i denote the execution time of task i . Prior studies typically assume an inverse linear relationship between execution time and CPU frequency. However, modern processors exhibit nonlinear behavior in practice. Thus, we obtain τ_i for each task empirically and use its values for the training of our proposed

algorithm. Thus, the total computation energy consumption of this task on big and LITTLE cores, denoted by $E_i^b(f)$ and $E_i^L(f)$, are given by:

$$E_i^b(f) = \lambda \tau_i P_d^b(f) + \rho \tau_i P_s^b(f), \quad (8)$$

$$E_i^L(f) = \lambda \tau_i P_d^L(f) + \rho \tau_i P_s^L(f), \quad (9)$$

where λ and ρ are constants that specify the contribution of dynamic and static power consumption to the total energy. These parameters are also determined by the fabrication process [26].

2) COMPUTATION OFFLOADING

When offloading computation tasks to the edge server, the energy consumption of the wireless interface must be considered. The communication energy required to transmit task i of size b_i to the edge server can be calculated as follows:

$$E_i^O(q_i) = q_i \frac{b_i}{r_i}, \quad (10)$$

where q_i and r_i denote the transmission power and the data rate of wireless interface, respectively, when offloading task i .

3) TOTAL ENERGY

We now obtain the total energy consumption required for task processing. We introduce three binary decision variables for each task $i \in 1, \dots, N$, denoted by x_i^O , x_i^b , and x_i^L , which represent offloading to the edge server, local processing on big cores, and local processing on LITTLE cores, respectively. Specifically, $x_i^O = 1$ indicates that task i is offloaded to the edge server, $x_i^b = 1$ signifies local execution on big cores, and $x_i^L = 1$ indicates local execution on LITTLE cores. To ensure that each task is scheduled for only one processor, we impose the constraint $x_i^O + x_i^b + x_i^L = 1$ for all $i \in 1, \dots, N$. The decision profile of different tasks of set $\mathcal{T}$ is defined as $\mathbf{X} = (\mathbf{x}^O; \mathbf{x}^b; \mathbf{x}^L)$, where vectors $\mathbf{x}^O = (x_i^O)_{i \in \mathcal{T}}$, $\mathbf{x}^b = (x_i^b)_{i \in \mathcal{T}}$, and $\mathbf{x}^L = (x_i^L)_{i \in \mathcal{T}}$. We also denote the operating frequency of big and LITTLE cores by f^b and f^L , respectively. Given the decisions x_i^O, x_i^b, x_i^L for task i , its energy consumption which is denoted as E_i^T , can be calculated as:

$$E_i^T(f^b, f^L, q_i) = x_i^b E_i^b(f^b) + x_i^L E_i^L(f^L) + x_i^O E_i^O(q_i). \quad (11)$$

E. PROBLEM FORMULATION

We now formulate the energy optimization problem. Our objective is to minimize the total energy consumption required to process all tasks in the set $\mathcal{T}$, while ensuring that their respective deadlines are met, inspired by the approaches in [14] and [35]. To ensure a task $i = 1, \dots, N$ meets its deadline, its actual execution time (AET) must be less than the specified deadline, as denoted by p_i . The optimization

problem can be formulated as follows:

$$\underset{\mathbf{X}, \mathbf{q}, f^b, f^L}{\text{minimize}} \quad \sum_{i=1}^N \left(x_i^b E_i^b(f^b) + x_i^L E_i^L(f^L) + x_i^O E_i^O(q_i) \right) \quad (12a)$$

$$\text{subject to} \quad f^b, f^L \in \mathcal{F}, \quad (12b)$$

$$\tau_i \leq p_i, \forall i = 1, \dots, N, \quad (12c)$$

$$x_i^b + x_i^L + x_i^O = 1, \forall i = 1, \dots, N, \quad (12d)$$

$$x_i^b, x_i^L, x_i^O \in \{0, 1\}, \forall i = 1, \dots, N. \quad (12e)$$

In this problem, the objective function in (12a) is defined to minimize the energy required to execute the task set. Constraint (12b) ensures that the selected frequencies are within the allowable set of values. Additionally, constraint (12c) enforces that AET is less than the task period, as it is not feasible for a task's execution time to exceed the period at which it is generated. In other words, it guarantees that each task is completed before its deadline. Finally, constraints (12d) and (12e) ensure that each task runs on only one core or is sent to the edge server.

The formulated optimization problem is a mixed-integer nonlinear program (MINLP) problem which is non-convex. Despite significant advancements in optimization methods, state-of-the-art solvers remain inefficient or even unable to solve general non-convex MINLPs to global optimality. In addition to the inherent computational burden of such combinatorial problems, we explicitly establish the NP-hardness of problem (12) in the following theorem.

Theorem 1: Problem (12) is NP-hard.

Proof: To prove Theorem 1, we show that an instance of problem (12) is NP-hard. Specifically, we construct a restricted version of the problem in which the CPU frequencies and transmission power levels are fixed and given in advance, while the core selection and offloading decisions remain as variables. We write the resulting problem as below where the constants c_i^b, c_i^L , and c_i^O are obtained by substituting the fixed frequency and power values into the corresponding terms in (11).

$$\underset{\mathbf{X}}{\text{minimize}} \quad \sum_{i=1}^N c_i^b x_i^b + c_i^L x_i^L + c_i^O x_i^O \quad (13a)$$

$$\text{subject to} \quad x_i^b + x_i^L + x_i^O = 1, \forall i = 1, \dots, N, \quad (13b)$$

$$x_i^b, x_i^L, x_i^O \in \{0, 1\}, \forall i = 1, \dots, N. \quad (13c)$$

The above problem can be reduced to the well-known partition problem, which is NP-complete [36]. This reduction shows the combinatorial scheduling problem alone is NP-complete even before accounting for the additional nonlinearities introduced by DVFS or the wireless rate function. Hence, problem (12) is at least as hard as an NP-complete problem and is thus NP-hard, which completes the proof. ■

The ultimate objective of our proposed algorithm is to minimize the long-term energy consumption averaged over time steps. Problem (12) is formulated for a certain time step which is omitted for the clarity of presentation.

The time step will be formally defined in Section IV. Beyond the inherent computational complexity of problem (12), this long-term objective, combined with the highly dynamic nature of mobile environments, makes solving the problem using classical optimization techniques even more challenging. To overcome these challenges, in the next section, we employ a model-free DRL algorithm to learn the optimal decision-making policy under dynamic conditions and propose an algorithm to achieve long-term energy minimization.

IV. IDEAS

Model-free DRL techniques are promising approaches for solving the task scheduling, power allocation, and offloading problem presented in (12). These methods facilitate decision-making based on local observations, eliminating the need to model the underlying dynamics of the system. To enhance the estimation of the expected long-term energy consumption in our proposed algorithm, we incorporate DQNs. This model-free and off-policy algorithm uses multilayer neural networks to estimate the value of each possible action, thereby optimizing the decision-making process. In this section, we first introduce key terminologies, and then propose our model-free DQN-based algorithm.

A. DQN

RL is a machine learning approach that learns the optimal policy over time through interaction with the environment. The agent makes decisions in the environment based on its observations of the current state. Through learning and exploration phases, the agent improves its policy by using feedback from the environment to converge toward optimal decision-making. At each step, the agent observes the state of the environment and selects an action from the action space. The environment then provides feedback in the form of a reward or penalty, which reflects the effectiveness of the agent's decision, and the system transitions to a new state. This process continues iteratively until the agent learns the optimal policy by maximizing cumulative rewards (or minimizing cumulative penalties).

Q-learning is one of the most well-known RL algorithms, aiming to improve the Q-value for all state-action pairs. Let (s, a) denote a state-action pair. Traditional Q-learning maintains a Q-table in which each entry (s, a) stores the Q-value, updated based on the agent's received reward. The function $Q(s, a)$ represents the expected discounted sum of future rewards when the agent takes action a in state s , followed by optimal actions in subsequent states.

To accurately model state transitions within DQN, we define discrete time steps $k = 1, 2, 3, \dots$. These time steps represent sequential decision points at which the proposed algorithm observes the current system state, takes an action, and transitions to the next state based on the chosen action and the environment's response. iDEAS evolves over these time steps and progressively learns the optimal decision-making policy through interaction with the

environment. It is important to note that problem (12) is formulated for a single time step to enhance clarity in problem representation. However, the proposed DQN operates over multiple time steps to capture the temporal dynamics of the system. We denote the set of tasks at time step k by $\mathcal{T}_k$. The length of each time step is defined as the least common multiple of the periods of the tasks within that step. At each time step, a set of tasks is generated, and iDEAS decides which tasks to offload, along with selecting the offloading power and the local processor's operating frequency.

We further define (s_k, a_k) as the state-action pair at time step k , where the Q-value is updated using

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha(R_{k+1} + \gamma \max_a Q(s_{k+1}, a) - Q(s_k, a_k)),$$

where α and γ represent learning rate and discount factor, respectively, and R_{k+1} denotes the reward value at time step $k + 1$.

Traditional tabular Q-learning suffers from instability, sample inefficiency, and a lack of generalization capability [37]. To address these limitations, DQN was introduced in [37], where the Q-function is approximated using a DNN. The DNN is trained over a minibatch of experience samples drawn from a replay buffer. The target value for the network is the Q-value, and the network weights are updated by minimizing the squared error between the predicted Q-value and the target Q-value using gradient descent.

The use of function approximators in DQN enables better generalization to unseen states. Unlike the tabular Q-learning approach, where each update affects only a single state-action pair, DQN leverages its approximation capabilities to influence a broader range of similar states during updates. Furthermore, the replay buffer improves convergence by breaking the temporal correlations between sequential transitions and increases data efficiency by allowing the reuse of past experiences during training. There are other variants of the DQN algorithm, such as Double DQN [38] and Dueling DQN [39], which improve the standard DQN by enhancing training stability and reducing overestimation bias. However, we use the base DQN due to its lower computational requirements and sufficiently strong performance compared to existing methods, which will be demonstrated in the next section.

B. STATE, ACTION, AND REWARD

We now define the states, actions, and reward structure used in our proposed iDEAS algorithm.

1) STATES

The state of the system at time step k consists of the following components:

- **Task utilization:** The utilization of tasks at time step k , represented as vector $\mathbf{u}_k = (u_i)_{i \in \mathcal{T}_k}$.

- **Task WCET:** The worst-case execution time for each task in $\mathcal{T}_k$, which provides an upper bound on execution duration, represented as $\mathbf{w}_k = (w_i)_{i \in \mathcal{T}_k}$.
- **Task size:** The input data size of tasks, which influences both offloading decision and processor allocation. We define vector $\mathbf{b}_k = (b_i)_{i \in \mathcal{T}_k}$ to represent these states at time step k .
- **Channel state:** The wireless channel gain during offloading, captured by $\mathbf{h}_k = (h_i)_{i \in \mathcal{T}_k}$.
- **CPU utilization:** The current utilization of the big and LITTLE CPU cores, denoted as u_k^b and u_k^L , respectively. These values are influenced by the previous states and actions.

We formally define the environment state at time step k as:

$$s_k = (\mathbf{u}_k, \mathbf{w}_k, \mathbf{b}_k, \mathbf{h}_k, u_k^b, u_k^L).$$

The DQN state is designed to reflect generated tasks and channel state to enable the RL agent to distinguish between various scenarios.

2) ACTIONS

In each state, the agent makes offloading decisions and sets the DVFS frequency. Specifically, the mobile device selects either the LITTLE core, big core, or edge server to execute each task. If a local device (i.e., LITTLE or big core) is selected, the frequency of the corresponding core is set. If the edge server is chosen (i.e., offloading the task), the transmission power of the wireless interface is set. To define the offloading and task allocation decisions for tasks $\mathcal{T}_k$ in each time step k , we introduce $\mathbf{x}_k^O = (x_i^O)_{i \in \mathcal{T}_k}$, $\mathbf{x}_k^b = (x_i^b)_{i \in \mathcal{T}_k}$, and $\mathbf{x}_k^L = (x_i^L)_{i \in \mathcal{T}_k}$, where x_i^O , x_i^b , and x_i^L are defined in Section III. Furthermore, we define $\mathbf{q}_k = (q_i)_{i \in \mathcal{T}_k}$ as the transmission power profile for all tasks in the set $\mathcal{T}_k$ at time step k . Let f_k^b and f_k^L denote the operating frequencies of the big and LITTLE cores at time step k , respectively. The agent's action at time step k is defined as:

$$\mathbf{a}_k = (\mathbf{x}_k^O, \mathbf{x}_k^b, \mathbf{x}_k^L, \mathbf{q}_k, f_k^b, f_k^L).$$

3) REWARD

We introduce a reward function for the agent that incorporates both energy consumption and execution latency. This reward function is designed to first motivate the agent (i.e., the mobile device) to take actions that satisfy task deadlines and then encourage energy reduction. To construct the reward function, we define a cost function for each task $t_i \in \mathcal{T}_k$ in time step k as follows:

$$C_i = E_i^T + \zeta^{\text{le}} \text{AET}_i, \quad \forall t_i \in \mathcal{T}_k, \quad (14)$$

where ζ^{le} is a latency-energy coefficient that balances the importance of AET with consumed energy. Moreover, E_i^T is the total energy consumption of task i , as given in (11). If a task misses its deadline, the cost is set to a high value to discourage such outcomes and facilitate gradient descent during the optimization for better convergence.

We define the reward function for time step k based on the cost function as below.

$$R_k = \sum_{t_i \in \mathcal{T}_k} e^{-\beta_r C_i}, \quad (15)$$

where β_r is a weighting factor emphasizing small cost values. A large β_r prompts the agent to focus on even minor improvements but may cause instability and greater sensitivity to small fluctuations during the convergence process. In the summation of (15), each term corresponds to the reward of task t_i , taken over all tasks in $\mathcal{T}_k$.

C. DQN-BASED ALGORITHM DEVELOPMENT

Our DQN method is presented in Algorithm 1. This algorithm begins by specifying the following parameters. The batch size in a DQN network specifies the number of data records processed in each training update. The target network update interval specifies the steps at which the target Q-network is updated, while the weight decay parameter in the Adam optimizer regulates model weights to prevent overfitting. Additionally, γ is the discount factor, and ϵ_{decay} determines the decay proportion of the ϵ parameter in the ϵ -greedy exploration strategy. Moreover, $\max_{\epsilon}$ and $\min_{\epsilon}$ represent the upper and lower bounds of the ϵ parameter, and ϵ_{update} indicates the update step for ϵ .

Prior to executing the algorithm, the main Q-network's weights are initialized with Xavier initialization [40]. The target Q-network's weights are initialized to match those of the main Q-network. In each episode iteration, a new set of tasks from $\mathcal{T}_k$ is generated. Based on these tasks, the next state is observed, and the corresponding action is selected. We employ the ϵ -greedy method for exploration. Tasks are classified into three distinct sets represented by $\mathcal{T}_k^O$, $\mathcal{T}_k^b$, and $\mathcal{T}_k^L$, according to the optimal action (Line 9), with each set allocated to its designated execution target. For local computing, the tasks are executed on their assigned cores with the operating frequency adjusted based on the action. For offloading to the edge server, the transmission power is specified by the chosen action. The reward is then computed, and the tuple $(s_k, \mathbf{a}_k, R_k, s_{k+1})$ is stored in the replay buffer for subsequent Q-network training (Line 16). A random batch is sampled from the replay buffer, and the Q-network is updated using the Adam optimizer with a smooth L1 loss function. Finally, if the update step for ϵ or the target Q-network is reached (Lines 19 and 21), they are updated accordingly.

D. COMPLEXITY ANALYSIS

We evaluate the time complexity of our proposed algorithm. iDEAS comprises a three-layer fully connected neural network. The time complexity of this network is $\mathcal{O}(\sum_{j=1}^L \eta_j \eta_{j-1})$, where $L = 3$ represents the three-layer multilayer perceptron, and η_j is the number of neurons in layer j . Thus, for an input size M , the time complexity of iDEAS is approximately $\mathcal{O}(64M + 2048 + 960) \approx \mathcal{O}(M)$.

Algorithm 1 iDEAS Algorithm

Input: Batch size, update target net, weight decay, target load, train iterations, discount factor γ , ϵ_{decay} , $\max_{\epsilon}$, $\min_{\epsilon}$, ϵ_{update} , β_r , ζ^{le} , $\mathcal{T}$

Output: a^* , Q^*

Initialize main Q network: $Q \leftarrow \text{Xavier}$;

Initialize target Q network from main Q network;

while $k < \text{train iterations}$ **do**

Generate random task $\mathcal{T}_k$ based on target load;

Update current state s_k ;

if $\text{rand}() \geq \epsilon$ **then**

Choose best action a_k with maximum Q;

else

Choose a random action a_k for exploration;

According to action a_k , split $\mathcal{T}_k$ into $\mathcal{T}_k^O$, $\mathcal{T}_k^b$, $\mathcal{T}_k^L$;

Offload $\mathcal{T}_k^O$ to the edge server;

Execute $\mathcal{T}_k^b$ on big core;

Execute $\mathcal{T}_k^L$ on LITTLE core;

Calculate cost C_i for each task $t_i \in \mathcal{T}_k$ using (14);

Calculate reward R_k for $\mathcal{T}_k$ using (15);

Observe next state s'_{k+1} ;

Add $(s_k, a_k, R_k, s'_{k+1})$ to replay buffer;

Select a batch of data of 'batch size' from replay buffer;

Update main Q network with smooth L1 loss;

if $\epsilon \in \text{update step}$ **then**

$\epsilon \leftarrow \max(\min_{\epsilon}, \epsilon - \epsilon_{\text{decay}} * (\max_{\epsilon} - \min_{\epsilon}))$;

if $\epsilon \in \text{update target network step}$ **then**

$Q_k \leftarrow Q$;

We also assess the memory footprint of iDEAS. This involves calculating the number of learnable parameters in the model, given by

$$64M + 64 \times 32 + 32 \times 30 = 3008 + 64M.$$

Thus, the memory requirement of iDEAS is $\mathcal{O}(3008 + 64M) \approx \mathcal{O}(M)$, where M is the input size.

V. PERFORMANCE EVALUATION

In this section, we evaluate the performance of our proposed algorithm, iDEAS, via numerical experiments. We further compare iDEAS with several baseline schemes, the RRLO algorithm proposed in [13], and the DRLDO algorithm proposed in [14]. This comparison is highly relevant, as iDEAS, RRLO, and DRLDO employ DVFS and computation offloading in modern mobile devices. These algorithms share the objective of minimizing energy consumption, leveraging RL to achieve this goal. These factors make RRLO and DRLDO ideal reference benchmarks for comparison.

A. SIMULATION PARAMETERS

We consider a mobile device with an ARM multicore processor that employs Cortex A57 as the big core and Cortex

A53 as the LITTLE core [9]. The specifications and related parameters of the big and LITTLE cores are given in Tables 1 and 2, respectively [23]. The first column of each table shows the possible frequencies (i.e., f), the second column represents the dynamic power, and the third column shows the power consumption of the respective processor core. we assume the edge server has a CPU frequency of 2.8 GHz. This value lies within the typical range of commercial server processors and serves as a reasonable representative baseline to capture the performance gap between resource-constrained mobile devices and edge servers [23].

To ensure a realistic evaluation of task characteristics and data distributions, we generate the tasks using the dataset introduced in [13]. Specifically, we define four real-time tasks t_1, t_2, t_3, t_4 , each characterized by parameters (p_i, b_i, w_i) , where p_i represents the period, b_i is the task input data size, and w_i denotes its WCET, as detailed in Table 3. Note that WCET values are defined with respect to the edge server's frequency. Thus, these values correspond to the worst-case execution times of each task when executed on the edge server. When the DVFS algorithm selects a lower operating frequency for the big or LITTLE cores, both AET and WCET are scaled inversely with the frequency, reflecting a longer execution time.

TABLE 1. Big Core Specifications [23].

Frequency (MHz)	Power (mW)	Energy Consumption (J)
384	459.2	4.77
480	494.9	4.25
632	622.4	4.03
767	704.1	3.86
863	784.7	3.82
959	857	3.71
1247	1199	4.03
1341	1408.2	4.42
1440	1714.4	4.99
1534	1867.4	4.35
1632	2102	5.38
1690	2142.9	5.41
1820	2460.4	5.73

We define two task sets, each comprising a distinct combination of the aforementioned tasks in [13].

- Task Set I: $\mathcal{T} = (t_1, t_2, t_3, t_4)$,
- Task Set II: $\mathcal{T} = (t_1, t_2, t_2, t_4)$.

Task Set I serves as a comprehensive baseline, encompassing all tasks listed in Table 3. Task Set II is designed to evaluate the offloading scheme's adaptability and efficiency by replacing a computationally intensive task (i.e., t_3) with a less demanding one. This setup enables the investigation of scenarios where offloading or local execution decisions significantly affect system performance and energy consumption. These task sets are employed for performance evaluation.

We generate a diverse set of tasks for training to ensure comprehensive exploration by the DQN-based learning algorithm. For each task characteristic, its value is randomly and uniformly sampled from the intervals specified in Table 4.

TABLE 2. LITTLE Core Specifications [23].

Frequency (MHz)	Power (mW)	Energy Consumption (J)
384	193.8	2.78
461	204.1	2.51
600	244.9	2.36
670	255.1	2.25
786	290.8	2.18
863	301	2.11
959	306	1.98
1239	428.6	2.26
1440	556	2.56

TABLE 3. Task Descriptions [13]. It is important to note that the utilization of tasks is varied from small to large values to analyse iDEAS under all load conditions, while the characteristic of each task is drawn from the parameters listed in this table.

Task	p_i	w_i	b_i
t_1	50	7	300
t_2	30	2	150
t_3	100	10	500
t_4	60	3	400

TABLE 4. Training Tasks Parameters.

p_i	w_i	b_i
[20, 120]	[2, 12]	[100, 500]

TABLE 5. Hyperparameter Values.

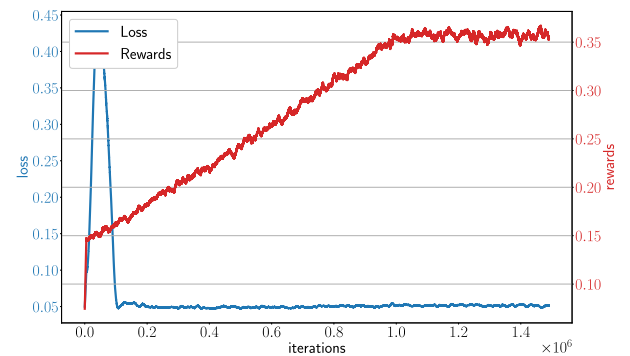
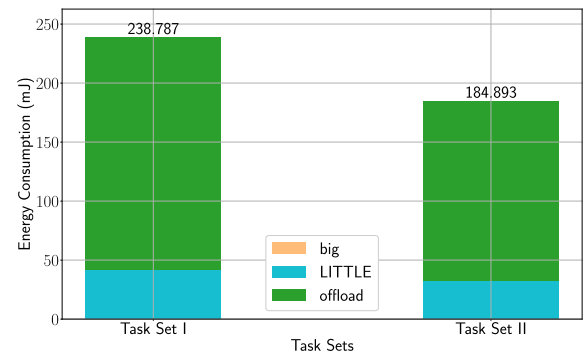
Parameter	Value
Training iterations	1.5×10^6
Evaluation iterations	10^4
Learning rate (α)	5×10^{-6}
Latency-energy coefficient (ζ^{le})	5×10^{-3}
Discount factor (γ)	0.95
Epsilon decay (ϵ_{decay})	0.001
Maximum epsilon ($\max_\epsilon$)	1.0
Minimum epsilon ($\min_\epsilon$)	0.02
Replay buffer size	10000
Update target Q-Network step	100
Batch size	32
Weight decay	1×10^{-3}
Reward coefficient (β_r)	0.05
Channel bandwidth	5MHz
Channel noise power (σ)	1×10^{-9}

At each training iteration, a set of four tasks is generated accordingly.

The hyperparameters used for training and evaluation are listed in Table 5. Note that we do not explicitly set the λ and ρ parameters in the CPU power consumption model. Instead, using Tables 1 and 2, we estimate total energy consumption (i.e., static and dynamic) based on the processor's frequency. This approach provides a more realistic estimate of energy consumption, as calculating λ and ρ parameters involves approximations that may not accurately reflect the processor's behavior. Further details on training and evaluation parameters are provided in [2].

B. EXPERIMENT: IDEAS PERFORMANCE ANALYSIS

In this subsection, we investigate the iDEAS characteristic and provide its numerical assessments. We begin by examining how the iDEAS loss and reward evolve over the time

**FIGURE 2.** The iDEAS loss and reward over time.**FIGURE 3.** Energy consumption levels for Task sets I and II. Our algorithm avoids allocating these tasks to the big cores as they can be more efficiently executed by alternative computing resources (i.e., LITTLE cores and edge server).

steps, as shown in Fig. 2. This figure illustrates the training performance of iDEAS, showing a rapid initial spike in loss followed by stable convergence, indicating effective policy learning. The steadily increasing reward curve confirms that the agent consistently improves its decision-making over time. The low and stable loss, combined with rising rewards, demonstrates that iDEAS effectively learns a task allocation policy that minimizes energy consumption, reaching convergence after an initial exploration phase.

We then evaluate the energy consumption of iDEAS for Task Sets I and II. Fig. 3 illustrates the energy consumption of the big core, LITTLE core, and task offloading to the edge server. As shown, iDEAS consumes less energy for Task Set II compared to Task Set I. Additionally, for Task Set II, the energy consumed by the LITTLE cores and wireless interface is lower than that for Task Set I. This is due to the lower computational intensity of tasks in Task Set II. These results indicate that the big cores are not heavily involved in processing either task set. iDEAS minimizes energy consumption by avoiding the use of the big cores as both task sets impose a low computational load on the mobile device. These results demonstrate that iDEAS dynamically adapts task execution to workload intensity, reducing energy consumption while maintaining task completion.

We next evaluate the performance of iDEAS across varying task loads. We vary task utilization to measure iDEAS's energy consumption and task drop percentage. The utilization

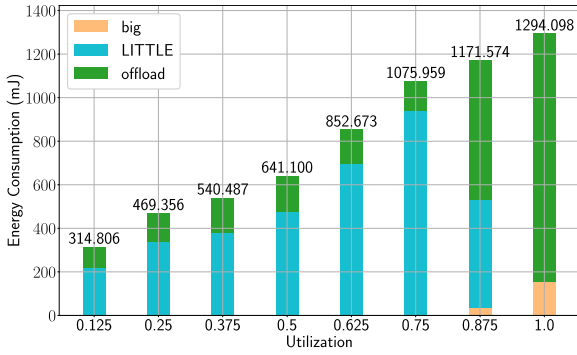


FIGURE 4. Energy consumption for various utilization values. At higher utilization levels, iDEAS begins allocating tasks to the big cores. Meanwhile, the energy spent for offloading the tasks decreases as more tasks are dropped since their deadlines cannot be met.

TABLE 6. iDEAS energy consumption improvement over other algorithms. Each entry in the table shows the percentage of energy saving achieved by iDEAS compared to the corresponding baseline algorithm.

Algorithm	iDEAS Improvement (Task Set I)	iDEAS Improvement (Task Set II)
Random	62.8%	59.75%
Local	81.13%	79.18%
Edge-only	84.26%	84.25%

of each task ranges from 0 to 1, and the total utilization is the accumulation of tasks' utilization values. Here, we normalize the total utilization by dividing it by the number of tasks to have a consistent analysis of results. Fig. 4 shows the energy consumption of the big and LITTLE cores, as well as the energy required for offloading tasks to the edge server. As depicted, iDEAS prioritizes assigning tasks to LITTLE cores over offloading at low utilization, while avoiding task allocation to big cores. This preference stems from the energy efficiency of LITTLE cores. At higher utilization, iDEAS shifts toward allocating more tasks to big cores. This occurs because LITTLE cores may fail to meet task deadlines under high task loads. Big cores, despite higher energy consumption, provide greater processing power to meet task deadlines. Across all utilization levels, iDEAS makes decisions that minimize energy consumption while ensuring task deadlines are met.

Fig. 5 shows the task drop percentage of iDEAS across varying utilization levels. We define a job's deadline as missed if it is not completed within its period. The task drop rate is calculated by dividing the number of tasks missing their deadlines by the total number of tasks. In this figure, dropped tasks are categorized into three groups: tasks assigned to LITTLE cores but not completed, tasks dropped by big cores, and tasks offloaded to the edge server but considered dropped due to missed deadlines. At utilization levels up to 0.375, iDEAS effectively allocates resources, resulting in minimal task drops. Beyond this utilization level, iDEAS have to drop some tasks, as no computing resources can complete them before their deadlines.

Overall, Figs. 4 and 5 demonstrate that iDEAS maintains energy efficiency at lower utilization levels with no task drops. As utilization increases, energy consumption rises,

and more tasks are dropped. This underscores the increasing trade-off between energy efficiency and task completion at higher utilization levels. Furthermore, as utilization increases and task drops become imminent, iDEAS assigns more tasks to the big cores and the edge server in the hope of completing them on time, while still aiming to minimize energy consumption.

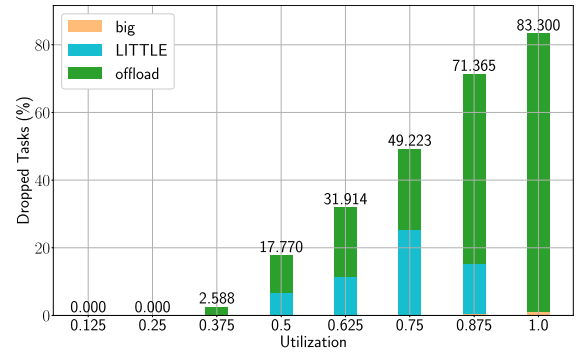


FIGURE 5. iDEAS task drop with respect to various utilization levels. When utilization reaches 1, tasks may still be executable by the edge server. However, the communication delay in offloading them results in missed deadlines, leading to higher task drops.

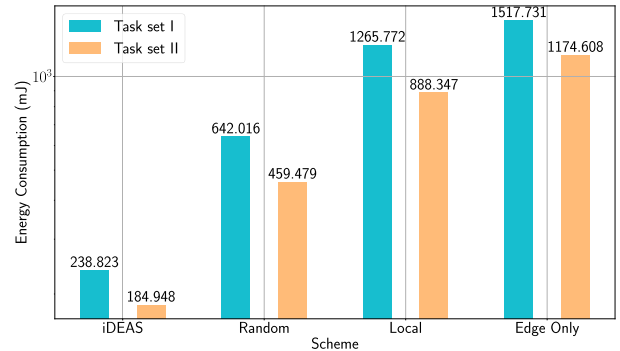


FIGURE 6. Energy consumption for four algorithms: (1) iDEAS, (2) random policy, (3) local computation only, and (4) edge-only execution. As can be observed, iDEAS outperforms the other three baseline schemes for both task sets.

C. EXPERIMENT: COMPARISON TO BASELINE SCHEMES

In this subsection, we compare the performance of iDEAS against three baseline schemes: Random policy, local computing, and edge-only execution. Fig. 6 shows the energy consumption of these algorithms. As shown, iDEAS consumes the least energy among the compared schemes. It outperforms the edge-only execution method by approximately 84.26% for both task sets, where all tasks are offloaded to the edge server. It also achieves 81.13% and 79.18% improvements over local computing for Task Sets I and II, respectively. Furthermore, iDEAS outperforms the random policy method by 62.8% and 59.75% for Task Sets I and II, respectively. Table 6 summarizes the performance improvements of iDEAS over the three baseline schemes for Task Sets I and II. These results highlight the adaptability of

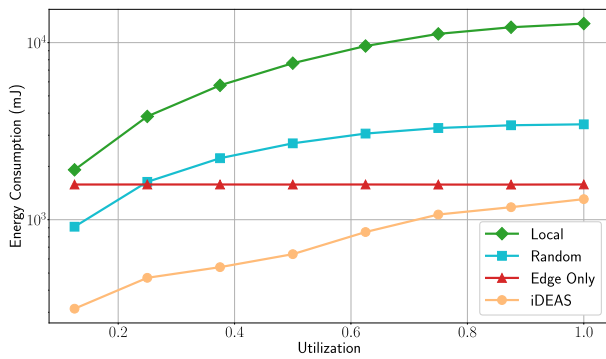


FIGURE 7. Energy consumption for four algorithms with respect to various utilization: (1) iDEAS, (2) edge-only execution, (3) random policy, and (4) local computation only. iDEAS outperforms the baseline schemes for all utilization levels.

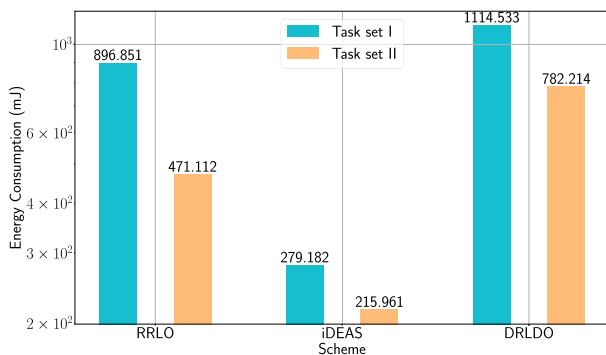


FIGURE 8. Comparison of energy consumption among iDEAS, RRLO [13], and DRLDO [14] on two task sets. iDEAS demonstrates significantly improved energy efficiency for both task sets.

iDEAS to Task Sets I and II, which differ in computational intensity.

We further evaluate the energy consumption of iDEAS across utilization levels in the range [0, 1], comparing it with baseline schemes. The results are shown in Fig. 7. This figure underscores the superiority of iDEAS over baseline schemes across various task utilization levels, showcasing its intelligent decision-making. Notably, the energy consumption of the edge-only execution scheme remains constant across utilization levels, as tasks have uniform size but varying computational intensity. Consequently, the energy required by the wireless interface to offload tasks to the edge server remains constant. This establishes an upper bound on iDEAS's energy consumption, as it increasingly prioritizes computation offloading at higher utilization levels in this scenario.

D. EXPERIMENT: COMPARISON TO RRLO AND DRLDO

To further demonstrate the performance of iDEAS, we compare it with two existing studies. The authors in [13] proposed RRLO, a double Q-learning method that determines CPU frequencies and employs a traditional DVFS algorithm for frequency tuning. DRLDO, proposed in [14], uses a DQN-based approach for offloading and DVFS decisions.

In this regard, DRLDO is closely related to our work; however, its state design does not incorporate channel conditions or task size information.

To have a fair comparison with the RRLO and DRLDO, we modify the iDEAS algorithm. Both of these algorithms assume a single-core CPU architecture. Thus, for this evaluation, we restrict iDEAS to use only the big cores for local execution, ensuring a fair comparison. RRLO uses either the cycle conserving (CC) or look ahead (LA) DVFS algorithm to adjust core frequencies and employs Q-tables to select offloading targets. DRLDO, on the other hand, jointly determines both offloading decisions and CPU frequency levels.

Fig. 8 illustrates the energy consumption of iDEAS compared to RRLO and DRLDO. As shown, iDEAS outperforms RRLO and DRLDO by approximately 69% and 75% for Task Set I, respectively. For the less computationally intensive Task Set II, iDEAS further reduces energy consumption by 54% and 72% compared to RRLO and DRLDO, respectively. These substantial improvements stem from RRLO's reliance on random Q-table updates, which can result in sub-optimal decisions, whereas iDEAS leverages intelligent and adaptive decision-making strategies to optimize energy efficiency. Moreover, DRLDO does not consider channel state and task information, resulting in inaccurate offloading decisions and degraded performance relative to RRLO and iDEAS.

We also evaluate the performance of iDEAS in comparison to RRLO and DRLDO across various task utilization levels. Fig. 9 depicts the energy consumption of these three schemes, while Fig. 10 presents their corresponding task drop rates. As shown in Fig. 9, iDEAS consumes significantly less energy than both RRLO and DRLDO, sustaining its advantage as utilization increases. Notably, iDEAS maintains a substantial performance advantage over RRLO and DRLDO, particularly at lower utilization levels. As depicted, RRLO's energy consumption increases steadily across all utilization levels. This is due to RRLO's higher task drop rates, as illustrated in Fig. 10, which stabilizes energy consumption at higher utilization levels. Similarly, DRLDO algorithm shows steady increase in energy consumption as the task utilization increases. Since DRLDO lacks feedback from the channel and environment, tasks are not properly offloaded, resulting in inefficient decisions and higher energy consumption.

Fig. 10 displays the task drop rates (in %) of all three algorithms, highlighting iDEAS's superior performance over RRLO and DRLDO in reducing task drops at high task utilization levels. RRLO starts dropping tasks at a utilization of 0.1, with a steeper rise in task drop rate compared to iDEAS. DRLDO, on the other hand, performs slightly better than both RRLO and iDEAS at lower utilization levels. However, as utilization increases, DRLDO fails to balance energy consumption and task drop rate, resulting in higher task drops than iDEAS and increased CPU energy consumption, as shown in Fig. 9. These results underscore iDEAS's superiority over RRLO and DRLDO in both energy

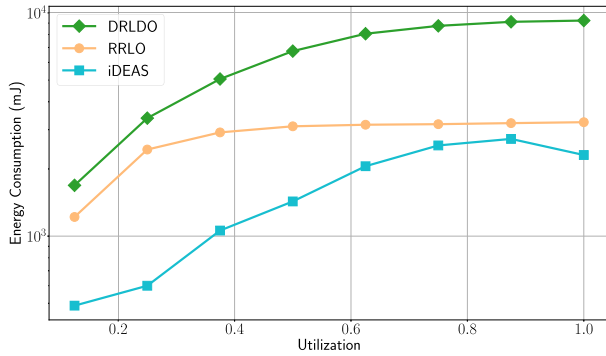


FIGURE 9. iDEAS energy consumption compared to RRLO [13] and DRLDO [14], when we vary the utilization level by increasing the number of tasks. As illustrated, iDEAS consistently achieves significantly lower energy consumption compared to both RRLO and DRLDO across all load levels.

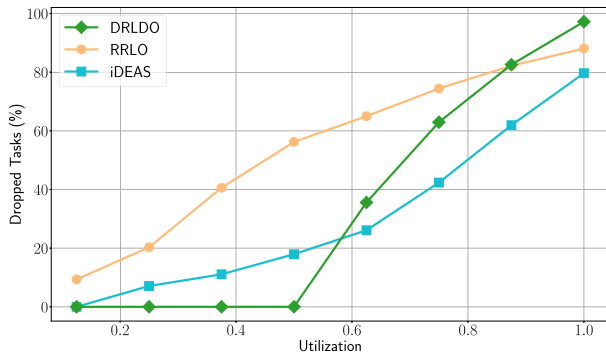


FIGURE 10. The corresponding task drop of iDEAS compared to RRLO [13] and DRLDO [14] with respect to various utilization.

efficiency and task reliability at higher utilization levels, owing to its effective frequency scaling and optimized offloading decisions.

VI. CONCLUSION

In this paper, we studied task scheduling in a mobile device with heterogeneous multicore big.LITTLE computing architecture that leverages DVFS. We proposed iDEAS, an algorithm designed to schedule the tasks and allocate them to the appropriate cores, adjust their frequencies, or offload them to an edge server. We first formulated a combinatorial optimization problem that aims to minimize the energy consumption of the mobile device subject to meeting deadline of all tasks in the highly dynamic mobile environment. To efficiently solve the problem, we then proposed a model free DQN-based algorithm. In particular, we designed a cost function that reflects both energy consumption and task processing delay, using which we introduced a reward function for various action choices and used DQN to estimate this reward and identify the most energy-efficient actions. We also analyzed the complexity and memory footprint of our proposed algorithm, demonstrating its feasibility for implementation. We further evaluated the performance of

our proposed algorithm through numerical experiments. Our results demonstrate that the proposed algorithm significantly reduces the mobile device's energy consumption. Additionally, the results show up to 75% reduction in energy consumption compared to two existing works in the literature, while simultaneously reducing the task drop.

Future work will focus on bridging the gap between simulation-based and real-world results, which may arise from factors such as observation latency and overhead, hardware non-idealities, and safety constraints during exploration. We will implement iDEAS as part of the Linux kernel to measure key dynamics and noise in real hardware and use lightweight profiling and statistical models to overcome latency and DVFS switching overhead challenges. We will also employ uncertainty-aware and safe RL techniques to enable reliable real-world deployment. Extending iDEAS to handle task dependencies is another direction for future work. We will additionally extend our proposed algorithm to consider task migration for greater flexibility and efficiency.

ACKNOWLEDGMENT

An earlier version of this paper was presented in part at the Proceedings of IEEE International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC), Toronto, Canada, in September 2023 [DOI: 10.1109/PIMRC56721.2023.10293955]. The algorithm source code is publicly available in [2].

REFERENCES

- [1] M. HajiKhodaverdian, H. Rastaghi, M. Saadat, and H. Shah-Mansouri, "Reinforcement learning-based task scheduling using DVFS techniques in mobile devices," in *Proc. IEEE 34th Annu. Int. Symp. Pers., Indoor Mobile Radio Commun. (PIMRC)*, Sep. 2023, pp. 1–6.
- [2] N. Samadi, F. Yadollahi, and H. Shah-Mansouri. (2025). *iDEAS Repository*. [Online]. Available: <https://github.com/NimaSamadi007/iDEAS-Implementation>
- [3] C. Baena, S. Fortes, O. S. Peñaherrera-Pulla, E. Baena, and R. Barco, "Gaming in the cloud: 5G as the pillar for future gaming approaches," *IEEE Commun. Mag.*, vol. 62, no. 11, pp. 76–82, Nov. 2024.
- [4] M. Kim, G. P. Nam, H. Kim, H. Park, and I.-J. Kim, "IG-FIQA: Improving classifiability-based face image quality assessment through intra-class variance guidance," *IEEE Access*, vol. 13, pp. 73987–73998, 2025.
- [5] Q. Wei, H. Wu, F. Shi, Y. Wan, and H. Ning, "A tutorial on meta-services and services computing in metaverse," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 16981–16995, May 2024.
- [6] K. Wang and A. Liu, "Accelerated constrained reinforcement learning based energy-efficient power scheduling algorithm for extended reality transmission," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Milan, Italy, Mar. 2025, pp. 1–6.
- [7] S. Govinda, B. Brik, and S. Harous, "A survey on deep reinforcement learning applications in autonomous systems: Applications, open challenges, and future directions," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 7, pp. 11088–11113, Jul. 2025.
- [8] S. K. Sahu, M. Gharbaoui, A. L. Ruscelli, G. Cecchetti, A. Sgambelluri, and P. Castoldi, "Remote medical support in emergency scenarios: A WebRTC-based solution," *IEEE Commun. Mag.*, vol. 63, no. 5, pp. 32–38, May 2025.
- [9] *Big.LITTLE Technology: The Future of Mobile*. Accessed: Jul. 31, 2025. [Online]. Available: <https://armkeil.blob.core.windows.net/developer/Files/pdf/white-paper/big-little-technology-the-future-of-mobile.pdf>
- [10] P. Pillai and K. G. Shin, "Real-time dynamic voltage scaling for low-power embedded operating systems," *ACM SIGOPS Operating Syst. Rev.*, vol. 35, no. 5, pp. 89–102, Dec. 2001.

- [11] L. Geng, H. Zhao, J. Wang, A. Kaushik, S. Yuan, and W. Feng, "Deep reinforcement learning based distributed computation offloading in vehicular edge computing networks," *IEEE Internet Things J.*, vol. 13, no. 14, pp. 12416–12433, Jul. 2023.
- [12] K. A. Alaghbari, H.-S. Lim, C. C. Zarakovitis, N. M. A. Latiff, S. H. S. Ariffin, and S. F. Chien, "Joint distributed computation offloading and radio resource slicing based on reinforcement learning in vehicular networks," *IEEE Open J. Commun. Soc.*, vol. 6, pp. 1231–1245, 2025.
- [13] H. Huang, Q. Ye, and H. Du, "Reinforcement learning based offloading for realtime applications in mobile edge computing," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Dublin, Ireland, Jun. 2020, pp. 1–6.
- [14] S. K. Panda, M. Lin, and T. Zhou, "Energy-efficient computation offloading with DVFS using deep reinforcement learning for time-critical IoT applications in edge computing," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 6611–6621, Apr. 2023.
- [15] Y. Tang, H. Huang, W. Zhan, G. Min, Z. Duan, and Y. Lei, "Battery-care resource allocation and task offloading in multi-agent post-disaster MEC environment," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2024, pp. 1–6.
- [16] C. Shi, J. Wei, Y. Zhu, and A. Schmeink, "Fog computing meets URLLC: Energy minimization of task partial offloading for URLLC services," *IEEE Access*, vol. 12, pp. 100328–100342, 2024.
- [17] H. Lv, Y. Guo, and L. Yang, "Study of energy-efficient scheduling in multi-core systems in dynamic voltage and frequency adjustment," *J. Phys., Conf. Ser.*, vol. 2033, no. 1, Sep. 2021, Art. no. 012143.
- [18] Q. Wang, X. Mei, H. Liu, Y.-W. Leung, Z. Li, and X. Chu, "Energy-aware non-preemptive task scheduling with deadline constraint in DVFS-enabled heterogeneous clusters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4083–4099, Dec. 2022.
- [19] D. Kim, Y.-B. Ko, and S.-H. Lim, "Energy-efficient real-time multi-core assignment scheme for asymmetric multi-core mobile devices," *IEEE Access*, vol. 8, pp. 117324–117334, 2020.
- [20] X. Li and G. Li, "An adaptive CPU-GPU governing framework for mobile games on big.LITTLE architectures," *IEEE Trans. Comput.*, vol. 70, no. 9, pp. 1472–1483, Sep. 2021.
- [21] J. Wang, S. Xiao, X. Guo, D. Jia, and W. Wu, "Energy-saving scheduling strategy for real-time applications on big.LITTLE architectures," *IEEE Internet Things J.*, vol. 12, no. 15, pp. 29460–29471, Aug. 2025.
- [22] S. Ki, G. Byun, K. Cho, and H. Bahn, "Co-optimizing CPU voltage, memory placement, and task offloading for energy-efficient mobile systems," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 9177–9192, May 2023.
- [23] Y. Geng, Y. Yang, and G. Cao, "Energy-efficient computation offloading for multicore-based mobile devices," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2018, pp. 46–54.
- [24] H. Rexha, S. Lafond, G. Rigazzi, and J.-P. Kainulainen, "Towards very low-power mobile terminals through optimized computational offloading," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, Dublin, Ireland, Jun. 2020, pp. 1–6.
- [25] H. Huang, M. Lin, Laurence. T. Yang, and Q. Zhang, "Autonomous power management with double-Q reinforcement learning method," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 1938–1946, Mar. 2020.
- [26] Q. Zhang, M. Lin, L. T. Yang, Z. Chen, and P. Li, "Energy-efficient scheduling for real-time systems based on deep Q-learning model," *IEEE Trans. Sustain. Comput.*, vol. 4, no. 1, pp. 132–141, Jan. 2019.
- [27] D. Ma and R. Bondade, "Enabling power-efficient DVFS operations on silicon," *IEEE Circuits Syst. Mag.*, vol. 10, no. 1, pp. 14–30, Mar. 2010.
- [28] A. B. Kahng, S. Kang, R. Kumar, and J. Sartori, "Enhancing the efficiency of energy-constrained DVFS designs," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 21, no. 10, pp. 1769–1782, Oct. 2013.
- [29] J. Zhang, R. Zheng, X. Zhao, J. Zhu, J. Xu, and Q. Wu, "A computational resources scheduling algorithm in edge cloud computing: From the energy efficiency of users' perspective," *J. Supercomput.*, vol. 78, no. 7, pp. 9355–9376, Jan. 2022.
- [30] H. Zhou, J. Wang, L. Zhao, D. Meng, G. Feng, and R. Li, "Joint optimization of charging time and resource allocation in wireless power transfer aided federated learning," *IEEE Internet Things J.*, vol. 12, no. 17, pp. 35065–35077, Sep. 2025.
- [31] J. Wang, H. Zhou, L. Zhao, D. Meng, and X. Shouzhi, "Joint optimization of charging time and resource allocation in wireless power transfer assisted federated learning," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Vancouver, BC, Canada, May 2024, pp. 1–6.
- [32] D. Meng, J. Guo, H. Zhou, Y. Zhang, L. Zhao, Y. Shu, and X. Fan, "Incentive-driven partial offloading and resource allocation in vehicular edge computing networks," *IEEE Internet Things J.*, vol. 12, no. 8, pp. 11023–11035, Apr. 2025.
- [33] M. Tang and V. W. S. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 1985–1997, Jun. 2022.
- [34] F. M. M. U. Islam and M. Lin, "Hybrid DVFS scheduling for real-time systems based on reinforcement learning," *IEEE Syst. J.*, vol. 11, no. 2, pp. 931–940, Jun. 2017.
- [35] M. Bolourian and H. Shah-Mansouri, "Energy-efficient task offloading for three-tier wireless-powered mobile-edge computing," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10400–10412, Jun. 2023.
- [36] M. Y. Kovalyov and E. Pesch, "A generic approach to proving NP-hardness of partition type problems," *Discrete Appl. Math.*, vol. 158, no. 17, pp. 1908–1912, Oct. 2010.
- [37] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [38] H. V. Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, Feb. 2016, pp. 2094–2100.
- [39] Z. Wang, T. Schaul, M. Hessel, H. V. Hasselt, M. Lanctot, and N. D. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, Jun. 2015, pp. 1995–2003.
- [40] S. K. Kumar, "On weight initialization in deep neural networks," 2017, *arXiv:1704.08863*.



NIMA SAMADI was born in Tehran, Iran, in 2000. He received the B.Sc. and M.Sc. degrees in electrical engineering from the Sharif University of Technology, Tehran, Iran, in 2023 and 2025, respectively. He is currently involved in research projects focused on edge computing and communication systems. His other areas of interests include computer vision, machine learning, and reinforcement learning.



FARBOD YADOLLAHI received the B.Sc. degree in electrical engineering from the Sharif University of Technology, in 2024. His research interests include energy-efficient computing, edge intelligence, and hardware–software co-design, with a focus on circuit and architecture-level techniques for embedded and mobile platforms.



HAMED SHAH-MANSOURI (Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees from the Sharif University of Technology, Tehran, Iran, in 2005, 2007, and 2012, respectively, all in electrical engineering. He was a Postdoctoral Research and Teaching Fellow with The University of British Columbia, Vancouver, Canada. He joined the Department of Electrical Engineering, Sharif University of Technology, in 2019, where he is currently an Assistant Professor. His research interests include the application of optimization and machine learning in computer networks and the Internet of Things.

...